

The efficiency of use of resources has two components, the use, the nominator, and the resources, the denominator. There are different ways of defining the denominator: the total resources required, the total resources available, the total resources pledged, or the total resources provisioned. The requirements drive the pledge process, while available resources include those beyond-pledged. Resources provisioned count the total resources actually acquired by the experiment through the provisioning system, in CMS's case the SI. Some of these quantities are well-defined. The request comes from the scrutiny process.

Description of the Computing Model (Requests)

The computing model and its simulation determine the resource requirements of CMS. The computing model is an abstraction of what computing resources (tape, disk, CPU) we need based on the workflow requirements to have a successful physics program, including detector data processing and reprocessing, Monte Carlo simulation production and reprocessing, and physics analysis. The inputs to the model are benchmarked CPU event timings (based on representative workflows), data placement and movement requirements (active disk storage needs), and archival storage needs (tape). The model is simulated with idealized provisioning to calculate the resource requirements. By construction, therefore, the model is 100% CPU and disk efficient.

Description of the Workflow Model

Step chains and task chains, resource requirement estimations of CPU count and memory (with uncertainties).

Description of the Provisioning Model

The CMS resource provisioning model ... *this is where reality confronts the idealized computing model.*

The ATLAS model is quite different. Input data sets are moved to the site where the processing step will take place before the job is started. The PanDa pilot is dimensioned for precisely one job in terms of CPU count and memory. Often there are many processing steps in a given workflow and these run at different sites, with different pilots, and intermediate data products are curated and moved according to need. Therefore, ATLAS requires more disk ([22% more than CMS](#) at Tier-1 and Tier-2 sites) and uses more network bandwidth ([5x CMS usage](#)) in order to store and move this data.

Brainstorming: In the old WLCG formula (written before the AI boom), 11 HS23 == 1 TB of disk space in terms of cost. (Since HS23 includes memory cost, which has ballooned along with disk prices, the ratio probably still holds to some approximation. Tape is now probably much cheaper on a relative basis, by a factor of 2-3.) The difference between the ATLAS and CMS disk requests is 80 PB, equivalent to 0.9 MS23 in cost. For comparison, the total world-wide CPU request of CMS is 4.55 MHS23, while ATLAS is 5.269 MHS23, or 0.719 MHS23 less than

ATLAS, for presumably comparable physics output (number of papers per year?) If CMS were to adopt an ATLAS-style model, our disk space requirements would likely increase to wipe out the equivalent difference in CPU requirements, on an equal-cost basis, and then some. Nevermind the hit to throughput.

Old ratios (2023?) 11 HS23 == 1 TB disk == 5 TB tape. Now perhaps 15 TB tape?

Tradeoffs between Efficiency and Throughput (Pareto Optimization)

Reusable pilots in CMS increase job throughput (can we measure this?) at a cost of a 2% CPU inefficiency, according to [older studies](#) (see Appendix C). Generally CMS sends jobs to where the input data sit (relevant only for processing and re-processing).

Table C-2: Summary of patterns leading to degradation of CPU efficiency, their estimated impact and short summary of the approach of CMS to mitigate them.

Source	Impact	Mitigation strategy
Application	10-20%	Reduce to the bare minimum all critical sections of the CMS applications, working on detector code, framework and collaborating with providers of external libraries. Continue to educate the analysis community about the impact their jobs have on the utilisation of resources, also providing concrete advice about how to improve.
High Memory Workflows	2%	Improve mechanisms to predict the memory usage of the application, reduce the memory needed by the application.
Remote Reads	1-5%	Balance as much as possible the tradeoff between disk and network needs.
Reusable Pilots	1-2%	Improve the scheduling system in collaboration with HTCondor and GlideinWMS developers.
Pilot Validation	1%	Optimise the 20 minute turnaround time before pilot resubmission without stressing the glidein infrastructure.
Site	<i>Varies</i>	Continue to reach out to our sites, giving priority to the ones providing more computing power (and therefore having a bigger impact on overall efficiency), in order to signal site-specific CPU efficiency degradations and help the local contacts in the debugging.

From: [Preliminary Resource Request for 2022](#) (September 2020), Appendix C.

Valid but messy workflows: high-memory workflows, inefficient but urgent workflows, ... what else?

If ATLAS needs to run a high-memory workflow, they request a high-memory pilot. While the CPU efficiency of the job may be measured to be very high relative to the resources provisioned in the pilot, the denominator does not capture the CPU idled on the worker node due to memory starvation. This is the difference between provisioned and available (but idled) CPU resources.

For CMS, the idled CPU is accounted for in the pilot. In the 2020 study this was approximately 2% averaged over the year.

Problem Definition (Sep. 9, 2026)

Start from resource requirements. An idealized model of what we need based on workflow requirements for physics, based on average event timings, data movements, archival and active storage needs.

In reality those workflows don't fit into perfectly-dimensioned, instantly instantiated resource "packets", but real pilots running on real worker nodes, with varying performance against the idealized benchmark, and workflow resource requirements that vary with respect to the average event timings in the model. Often the workflow requests are wrong, or vary so greatly from site to site that the dispersion is significant.

Strategic goals of CMS O&C (2020-2024):

<https://cms-docdb.cern.ch/cgi-bin/DocDB/ShowDocument?docid=14639>

Contribution to O&C Weekly Meeting (Sep. 2, 2026)

Indico: <https://indico.cern.ch/event/1722066/>

Reviewers of CMS have focused on CPU and disk resource utilization efficiency over the past several years, and in response to their recommendations CMS has made incremental improvements in the efficiencies, sometimes with regressions in later years. Maximizing computing utilization over several classes of resources simultaneously, for example CPU, memory, disk, and networking is what is called in other fields (such as economics) a Pareto optimization problem. In such optimizations, increasing the efficiency of one variable often results in a decrease in efficiency or utilization of another, such as the tradeoff between CPU efficiency (lowered) and disk space utilization (improved) when reading pre-mix pileup libraries remotely. Another example is using a fixed CPU core count for step-chain workflows (which will be more inefficient for certain steps) which improves network and disk utilization (the lack of need to transfer and curate intermediate data products). The real metric that we are interested in, however, our figure of merit, is not any of these. It is workflow throughput in wallclock time, though balanced against (monetary or environmental) cost.

Since CMS explicitly requests CPU and disk capacity (but not directly memory or networking) through a scrutiny process, the reviewers focus on the efficient use of those resources in and of themselves and not others, without looking at the amount of work being accomplished, which is difficult to evaluate. We can presume they will also focus on efficient use of GPUs when CMS will request them, which will further complicate the Pareto optimization problem. For example,

offloading work to GPUs will presumably worsen CPU efficiency, but also speed up workflow completion times (our figure of merit). Otherwise why would we use GPUs? How will GPU efficiency even be measured?

However, the reviewers are not wrong. If we are not using a resource efficiently, then we perhaps asked for too much. They don't expect perfection, but there are minimum levels below which they get uncomfortable. That level for CPU utilization is 80%. The pain threshold for *active* disk utilization is higher, around 90%. Unfortunately we as an experiment need to be respectful of these boundary conditions while also focusing on our figure of merit to accomplish the greatest amount of work during the year, or to accomplish important and urgent work from a physics point of view regardless of the CPU efficiency.

If we accept these boundary conditions (and I think we don't have a choice), what problems would we like to solve, looking at the challenges rather than the solutions? Some possibilities (please think of some more!):

1. Different steps in a workflow most efficiently use different CPU core counts, a challenge that will only be compounded with GPU workflows since it becomes a two-dimensional optimization problem.
2. Workflow applications (payloads) have low CPU efficiency
3. Workflow applications (payloads) have low GPU efficiency
4. Workflow applications have high failure rates, independent of sites
5. Sites have high job failure rates
6. Workflows have inaccurate resource request levels, resulting either in mass failures (badput) if they are too low or wastage (inefficiency) if they are too high.
7. Certain resources in pilots (CPU, GPU, memory) are underutilized. (Backfilling overcommitted pilots has helped with this already.)
8. The HTCondor collector (scheduler) struggles to keep up with heavy load, resulting in idle resources. (While we don't face this problem at the moment, we have in the not-so-distant past, so imagining this could become a problem again is not unrealistic.)
9. Is IaaS or local provisioning of GPUs more efficient? Can we measure this?

Thinking of some possible directions for solutions to these problems, though some may have been solved already in other ways:

- The first challenge has been proposed to be solved at the framework level, but are there solutions in the provisioning layer?
- Challenges #2-#6 have to be handled manually now after alarms are raised, or have static limits imposed. Are there agentic AI operator solutions? Could such an AI operator adjust workload requirements on the fly, or actively suspend failing workflows or sites?
- For #7-#8, could an agentic-AI collector more efficiently schedule workflows including adding backfill where it would help, rather than relying on static backfill algorithms and python loops over pending jobs and available pilots to schedule jobs one at a time?

kinds of technical capabilities in our WM could be useful while still adhering to our ultimate goal of accomplishing the greatest amount of work?

- Some of these we do already at the HTCondor/GlideinWMS level, such as overcommitting CPU by backfilling with single-core analysis jobs.
- Using agentie AI to ramp up and refine the resource requirements of workflows (`condor_gedit`) based on actual utilization. One could also suspend failing workflows or failing sites (and resubmit) with an agent.
- HTCondor schedules jobs on slots one by one in a loop, but realistically there are not that many different workflows with different requirements (autoclusters) and not that many slot types with the same CPU count and memory per core. Could an agent do bulk scheduling faster than a python loop over all jobs and slots? Typically there are only hundreds of job and slot autoclusters, vs. hundreds of thousands of individual jobs and pilot slots.
- Benchmarking the efficiency of using GPUs with IaaS vs. local use could be enlightening.
- Solving the problem that different steps in a workflow most efficiently use different CPU core counts.

An example of an optimization choice we made in order to maximize throughput was moving the majority of our workflows to step chains, which lowered disk and network needs but decreased the CPU efficiency of the application. Mitigating that reduction by overcommitting GPU puts more pressure on disk resources. Scheduling is a classic Pareto optimization problem where improving efficiency in one dimension often reduces it in another. A counter example is another experiment that relies completely on task chains, and is much more CPU efficient than CMS since workflows of the various steps can be tailored for the computing resources available for that task. The costs, however, are much greater disk requirements and significantly lower event processing and simulation times. This, of course, is something we should never say to reviewers. But what is better, to use CPU inefficiently at 70%, or use it at 100% but for twice as long wallclock time to achieve the same amount of work?

Therefore, I think we in CMS have to be clear what the figure of merit is here. It should likely continue to be workflow throughput, since that is what impacts data taking and physics. We have to accept that utilization of the computing resources in the multiple dimensions will be inefficient in at least some of them. Adding GPU into the mix will complicate the optimization problem even further. Using both CPU and GPU efficiently while maximizing throughput is likely an impossible task. We can continue to make incremental improvements to resource utilization efficiencies to get above thresholds not considered problematic by reviewers.

How to sell the overall strategy to reviewers? There is nothing else to compare it to within CMS, since the “road not chosen” wasn’t implemented e.g., a more CPU efficiency but lower

throughput workflow. Perhaps one could plot event throughput per CPU core wallclock hour vs. CPU efficiency to show the tradeoff over time? We should have this information in the request documents.

Data at our disposal: MC and data processing event timings (average), total CPU core years devoted to T0 and MC production worldwide. The quotient gives the event throughput per year = events/second / CPU core years. Plot that figure of merit against CPU efficiency, average disk space devoted to T0 or MC production from Rucio, over several years to get the Pareto curve?

Hmm Not so easy

Badput:

More failure-aware WM? WM that ramps up workflows slowly at first, checking success rates per site and adjusts accordingly? Agentic-AI monitoring of failing workflows?